МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Вычислительная математика»

Тема: Метод Ньютона

Студент гр. 4342 _____ Тиссен М.Ю.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Используя программы-функции NEWTON и Round из файла methods.cpp, найти корень уравнения f(x) = 0 с заданной точностью Eps методом Ньютона, а также исследовать скорость сходимости и чувствительность метода к ошибкам округления.

Задание

В случае, когда известно хорошее начальное приближение решения уравнения f(x) = 0, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$, сходящейся к корню уравнения f(x) = 0. Достаточные условия сходимости метода формулируются теоремой, приведенной в [1,7].

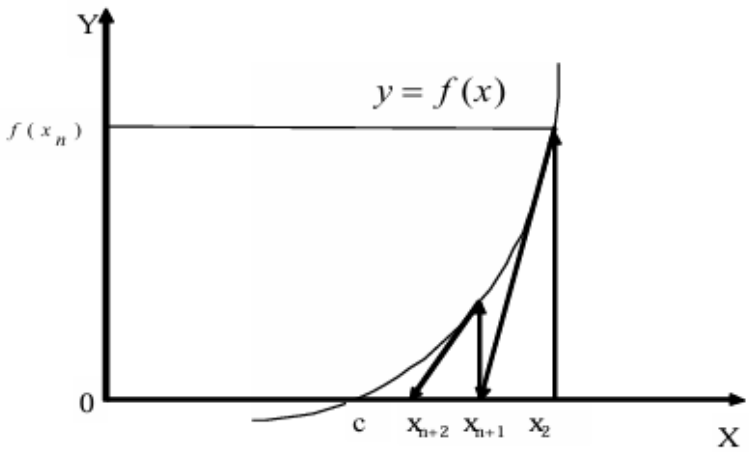

Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью Ox будет очередным приближением xn+1 корня уравнения f(x) = 0.

Для оценки погрешности n-го приближения корня предлагается пользоваться неравенством

$$|c - x_n| \leq \frac{M_2}{2m_1}|x_n - x_{n-1}|^2,$$

где M2-наибольшее значение модуля второй производной $|f''(x)|$ на отрезке [a,b]; m1 наименьшее значение модуля первой производной $|f'(x)|$ на отрезке [a,b]. Таким образом, если $|x_n - x_{n-1}| < \varepsilon$, то $|c - x_n| \leq \frac{M_2\varepsilon}{2m_1}$. Это означает, что при хорошем начальном приближении кор ня после каждой итерации число верных десятичных знаков в очередном приближении удваи вается, т.е. процесс сходится очень быстро (имеет место квадратическая сходимость). Из ука занного следует, что при необходимости нахождения корня с точностью итерационный про цесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon = \sqrt{2m_1\varepsilon/M_2}. \quad (3.1)$$

Рассмотрим один шаг итераций. Если на (n-1)-м шаге очередное приближение xn-1 не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1})$, $f'(x_{n-1})$ и сле дующие приближение корня

$x_n = x_{n-1} - f(x_{n-1})/f'(x_{n-1})$ При выполнении условия (3.1) величина xn принимается за приближенное значение корня с, вычисленное с точностью . В лабораторной работе № 5 предлагается, используя программы-функции NEWTON и ROUND из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения f(x) = 0 с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода. Для данной работы вид функции f(x) задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №5.

1) Графически или аналитически отделить корень уравнения f(x) = 0 (т.е. найти отрезки [Left, Right] , на котором функция f(x) удовлетворяет условиям сходимости метода Ньютона).

2) Составить подпрограммы - функции вычисления $f(x),\ f'(x)$, предусмотрев округление их значений с заданной точностью Delta.
3) Составить головную программу, вычисляющую корень уравнения f(x) = 0 содержащую обращение к подпрограммам , $f(x),\ f'(x)$, Round, NEWTON и индикацию результатов.
4) Выбрать начальное приближение корня x0 из [Left, Right] так, чтобы $f(x)\ f''(x)$>0.!!!!!!!
5) Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Для приближенного вычисления корней уравнения f(x) = 0 методом Ньютона предназначена программа - функция NEWTON, текст которой представлен в подразделе 3.7.

Выполнение работы

1. Отделение корня и выбор начального приближения

Рассматривается функция $f(x) = e^{(1/x^2)} - \ln(x)$, область определения которой задаётся условием $x > 0$. На выбранном интервале [3; 4] функция непрерывна. Значения на концах интервала имеют разные знаки:

$$f(3) \approx 0.0189067801 > 0, \qquad f(4) \approx -0.3217999022 < 0.$$

Следовательно, на отрезке [3; 4] существует корень уравнения $f(x) = 0$. Для метода Ньютона дополнительно проверяются производные функции. В работе используются выражения

$$f'(x) = -2e^{(1/x^2)}/x^3 - 1/x,$$

$$f''(x) = (x^4 + 6x^2e^{(1/x^2)} + 4e^{(1/x^2)}) / x^6.$$

На области $x > 0$ выполняется $f'(x) < 0$ и $f''(x) > 0$. Поэтому функция убывает и является выпуклой вверх. В качестве начального приближения выбрано $x_0 = 3$, так как выполняется условие сходимости $f(x_0)*f''(x_0) > 0$.

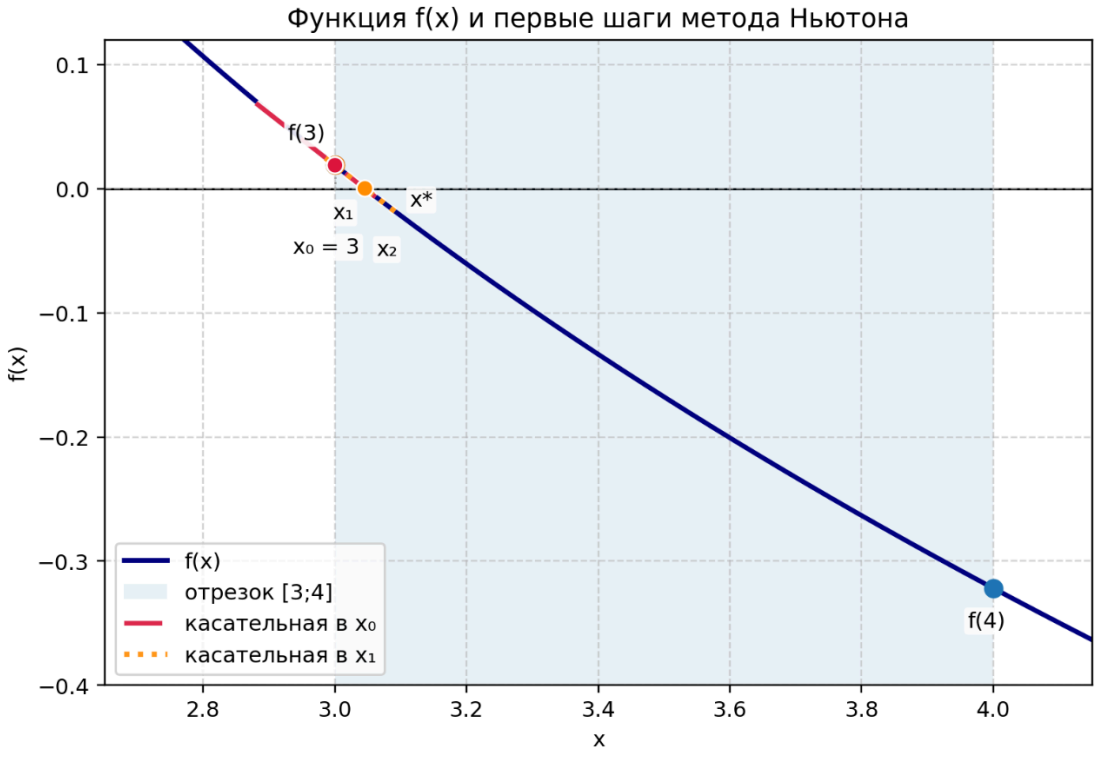

Рисунок 2 — График функции, отрезок изоляции [3; 4] и первые шаги метода Ньютона

2. Теоретические оценки для критерия остановки

В реализации используется оценка погрешности метода Ньютона. На отрезке [3; 4] берутся константы

$$m_1 = \min |f'(x)| = |f'(4)| \approx 0.2832654518,$$

$$M_2 = \max |f''(x)| = |f''(3)| \approx 0.2000220933.$$

Итерационный процесс прекращается, когда разность двух соседних приближений становится меньше величины

$$|x_n - x_{n-1}| < \varepsilon_0 = \sqrt{(2m_1 Eps / M_2)}.$$

Такой критерий соответствует теоретической оценке погрешности и используется в функции NEWTON напрямую.

3. Описание программной реализации

В файле main.cpp заданы функции f(x), f′(x) и f″(x) для варианта 18. В случае $x \le 0$ программа завершается, так как логарифм не определён.

В файле methods.cpp реализованы:

- функция Round(X, Delta), округляющая число до ближайшего значения, кратного Delta;
- функция SetRoundDelta(Delta), задающая используемую точность округления;
- функция SetNewtonConstants(m_1, M_2), передающая в метод Ньютона значения m_1 и M_2;
- функция NEWTON(X, Eps, N), выполняющая итерации $x_{n+1} = x_n - f(x_n)/f'(x_n)$.

На каждом шаге метода сначала вычисляются $f(x_n)$ и $f'(x_n)$, затем при необходимости они округляются с помощью Round. После этого находится следующее приближение. Если производная обращается в нуль, точность Eps задана неверно, либо очередное приближение выходит из области определения, программа аварийно завершается с сообщением об ошибке.

4. Результаты исследования сходимости

Сначала был найден опорный корень при Eps = 10^{-12} и Delta = 0:

$$x_ref \approx 3.045936104786175, \quad N = 3.$$

Далее метод Ньютона запускался для разных значений Eps без округления функции и производной. Результаты приведены в таблице 1.

Eps	**Число итераций N**	**Найденный корень**
1e-1	1	3.0454367004
1e-2	1	3.0454367004
1e-3	1	3.0454367004
1e-4	2	3.0459360463
1e-5	2	3.0459360463
1e-6	2	3.0459360463
1e-8	3	3.0459361048
1e-10	3	3.0459361048
1e-12	3	3.0459361048

Таблица 1 — Зависимость числа итераций метода Ньютона от точности Eps

По таблице видно, что метод Ньютона сходится очень быстро: для Eps от 10^{-1} до 10^{-3} хватает одной итерации, для Eps от 10^{-4} до 10^{-6} требуется две итерации, а для более строгой точности — три. Это подтверждает высокую скорость сходимости метода при удачно выбранном начальном приближении.

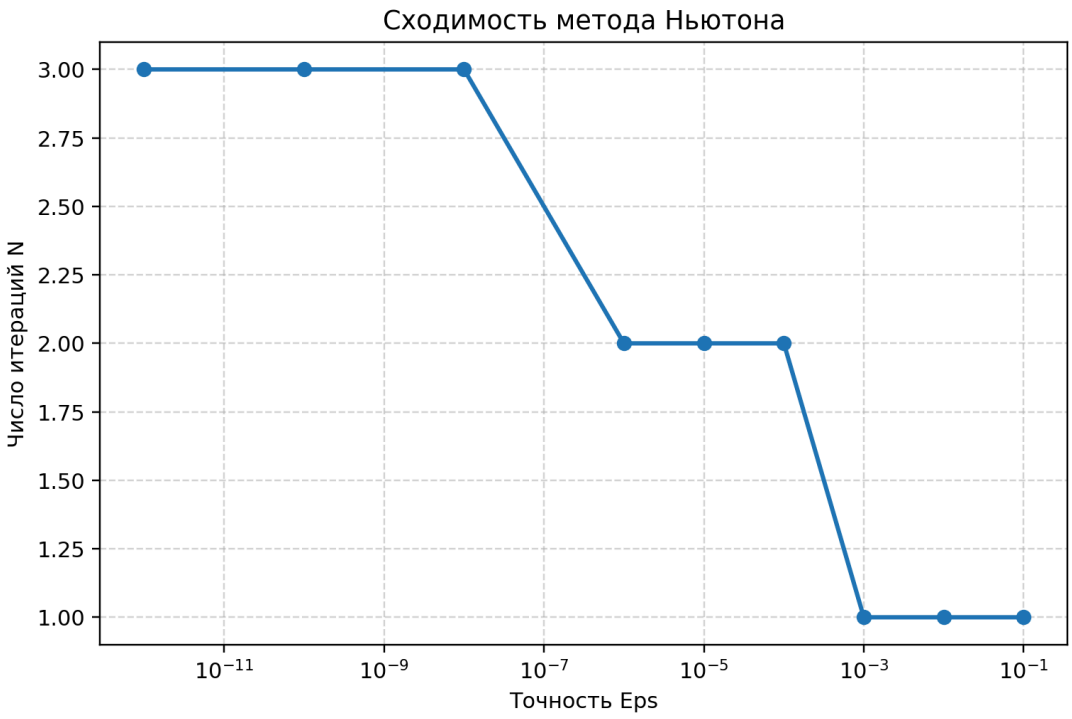

Рисунок 3 — Зависимость числа итераций от точности Eps

5. Исследование чувствительности к ошибкам округления

При фиксированной точности Eps = 10^{-8} параметр Delta изменялся от 0 до 10^{-1}. Округление применялось к значениям f(x) и f′(x) на каждом шаге метода. Абсолютная ошибка вычислялась как |x − x_ref|. Результаты сведены в таблицу 2.

Delta	**N**	**Найденный корень**	**|x − x_ref|**
0	3	3.0459361048	0.0000000000
1e-1	1	3.0000000000	0.0459361048
1e-2	2	3.0476190476	0.0016829428
1e-3	2	3.0456730769	0.0002630279
1e-4	3	3.0459129327	0.0000231720
1e-5	3	3.0459358371	0.0000002677
1e-6	3	3.0459356679	0.0000004368

Таблица 2 — Влияние параметра Delta на результат метода Ньютона

При Delta = 0.1 уже после первой итерации метод даёт x = 3.0, и ошибка заметно возрастает. По мере уменьшения Delta найденный корень стабилизируется и приближается к опорному значению. Это означает, что метод Ньютона чувствителен к грубому округлению, но при малых значениях Delta сохраняет высокую точность.

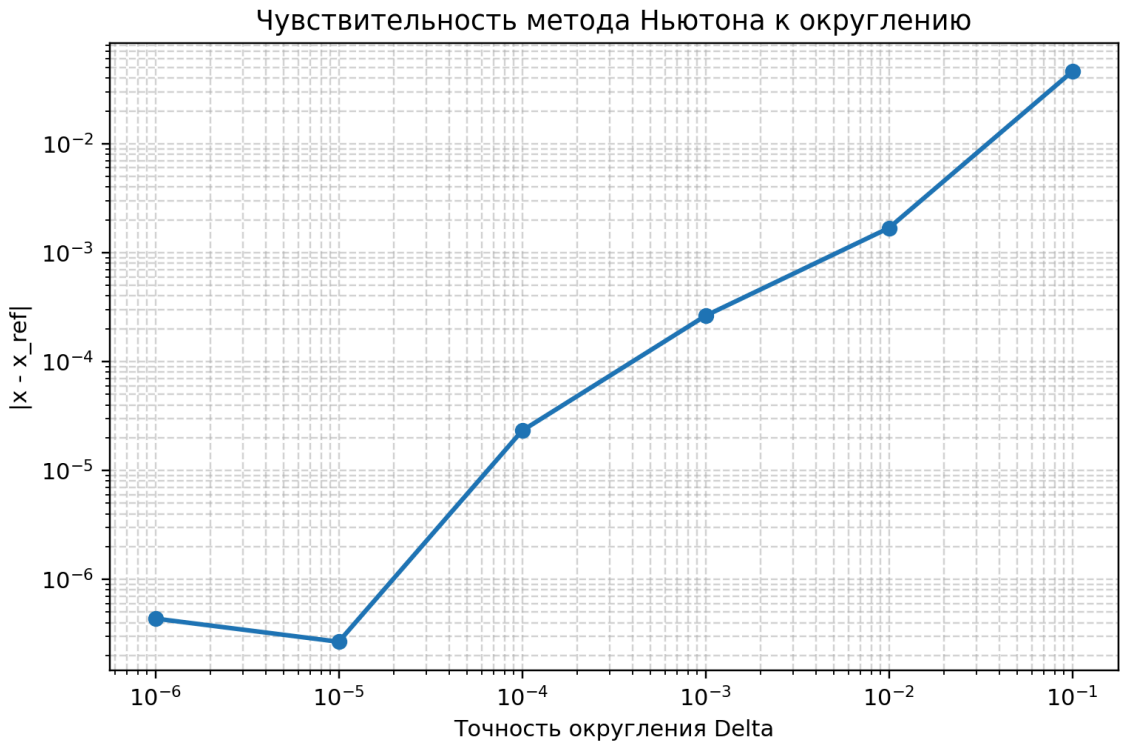

Рисунок 4 — Зависимость абсолютной ошибки от точности округления Delta

Вывод

В лабораторной работе реализован метод Ньютона для решения уравнения $e^{(1/x^2)} - \ln(x) = 0$. Аналитически и графически показано, что корень находится на отрезке [3; 4], а условие $f(x_0)*f''(x_0) > 0$ позволяет выбрать начальное приближение $x_0 = 3$. Реализация метода показала квадратичную скорость сходимости: при разумном начальном приближении для достижения высокой точности достаточно 2–3 итераций. Исследование чувствительности показало, что метод резко теряет точность при грубом округлении значений функции и производной, но при малом Delta остаётся устойчивым.

ПРИЛОЖЕНИЕ А

Название файла main.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"

// ============================================================
// ВАРИАНТ 18: f(x) = e^(1/x^2) - ln(x)
// Область определения: x > 0
// Интервал изоляции: [3; 4]
// ============================================================
double F(double x)
{
    if (x <= 0.0) {
        printf("Ошибка: x должен быть > 0\n");
        exit(1);
    }
    return exp(1.0 / (x * x)) - log(x);
}

double DF(double x)
{
    if (x <= 0.0) {
        printf("Ошибка: x должен быть > 0\n");
        exit(1);
    }
    return -2.0 * exp(1.0 / (x * x)) / (x * x * x) - 1.0 / x;
}

double DDF(double x)
{
    if (x <= 0.0) {
        printf("Ошибка: x должен быть > 0\n");
        exit(1);
```

```
    }
    return (pow(x, 4.0) + 6.0 * x * x * exp(1.0 / (x * x)) + 4.0 *
exp(1.0 / (x * x))) / pow(x, 6.0);
}

// ============================================================
// Головная программа
// ============================================================
int main()
{
    double Left = 3.0;
    double Right = 4.0;
    // Начальное приближение выбираем из условия f(x0) * f''(x0) > 0.
    double X0 = 3.0;
    double RefRoot, Root;
    double Eps, Delta;
    double m1, M2;
    int N;

    printf("========================================\n");
    printf("  Laboratornaya rabota #5: Metod Newtona\n");
    printf("  Variant 18: f(x) = e^(1/x^2) - ln(x)\n");
    printf("========================================\n\n");

    printf("Interval otdeleniya kornya: [%.2f; %.2f]\n", Left, Right);
    printf("F(Left)   = %.10f\n", F(Left));
    printf("F(Right)  = %.10f\n", F(Right));
    printf("DF(Left)  = %.10f\n", DF(Left));
    printf("DF(Right) = %.10f\n", DF(Right));
    printf("DDF(Left) = %.10f\n", DDF(Left));
    printf("DDF(Right)= %.10f\n\n", DDF(Right));

    // Оценки min |f'(x)| и max |f''(x)| на отрезке [3, 4].
    m1 = fabs(DF(Right));
    M2 = fabs(DDF(Left));
    SetNewtonConstants(m1, M2);
```

```
    printf("m1 = min |f'(x)| = %.10f\n", m1);
    printf("M2 = max |f''(x)| = %.10f\n", M2);
    printf("Nachalnoe priblizhenie x0 = %.10f\n\n", X0);

    // ==============================================================
    // Нахождение опорного корня с высокой точностью
    // ==============================================================
    SetRoundDelta(0.0);
    RefRoot = NEWTON(X0, 1e-12, N);
    printf("Opornyy koren (Eps=1e-12, Delta=0): %.15f (iteraciy:
%d)\n\n", RefRoot, N);

    // ==============================================================
    // Исследование скорости сходимости
    // ==============================================================
    printf("Zavisimost chisla iteraciy ot Eps:\n");
    printf("Eps\t\tIteracii\tKoren\n");
    printf("-----------------------------------------------\n");

    double EpsValues[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-8,
1e-10, 1e-12};
    int nEps = sizeof(EpsValues) / sizeof(EpsValues[0]);

    FILE* feps = fopen("newton_iterations_vs_eps.csv", "w");
    fprintf(feps, "Eps,Iterations,Root\n");

    for (int i = 0; i < nEps; i++) {
        Eps = EpsValues[i];
        SetRoundDelta(0.0);
        Root = NEWTON(X0, Eps, N);
        printf("%.1e\t%d\t\t%.15f\n", Eps, N, Root);
        fprintf(feps, "%e,%d,%.15f\n", Eps, N, Root);
    }
    fclose(feps);
    printf("\nDannye sohraneny v newton_iterations_vs_eps.csv\n");
```

```
    // ==============================================================
    // Исследование чувствительности к ошибкам округления
    // ==============================================================
    printf("\nChuvstvitelnost metoda (Eps=1e-8):\n");
    printf("Delta\t\tIteracii\tKoren\t\t\tError\n");

printf("-------------------------------------------------------------------
---------\n");

    double DeltaValues[] = {0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6};
    int nDelta = sizeof(DeltaValues) / sizeof(DeltaValues[0]);

    FILE* fdel = fopen("newton_sensitivity_vs_delta.csv", "w");
    fprintf(fdel, "Delta,Iterations,Root,Error\n");

    for (int i = 0; i < nDelta; i++) {
        Delta = DeltaValues[i];
        SetRoundDelta(Delta);
        Root = NEWTON(X0, 1e-8, N);
        double Err = fabs(Root - RefRoot);
        printf("%.1e\t%d\t\t%.15f\t%.15f\n", Delta, N, Root, Err);
        fprintf(fdel, "%e,%d,%.15f,%.15f\n", Delta, N, Root, Err);
    }
    fclose(fdel);
    printf("\nDannye sohraneny v newton_sensitivity_vs_delta.csv\n");

    return 0;
}
```

Название файла methods.h

```
#ifndef METHODS_H
#define METHODS_H

#include <cmath>

// Внешние функции варианта задаются в main.cpp.
```

```
extern double F(double x);
extern double DF(double x);
extern double DDF(double x);

// Установка точности округления значений функции и производной.
// Если Delta = 0, округление не выполняется.
void SetRoundDelta(double Delta);

// Функция Round - округление числа X до ближайшего значения, кратного
Delta.
double Round(double X, double Delta);

// Установка констант для критерия остановки метода Ньютона.
void SetNewtonConstants(double m1, double M2);

// Метод Ньютона для поиска корня уравнения F(x) = 0.
// X   - начальное приближение корня
// Eps - требуемая погрешность вычисления корня
// N   - число выполненных итераций
// Округление значений функции и производной задается через
SetRoundDelta.
// Критерий остановки: |x_n - x_(n-1)| < sqrt(2*m1*Eps/M2)
double NEWTON(double X, double Eps, int &N);

#endif
```

Название файла methods.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"

static double gDelta = 0.0;
static double gM1 = 0.0;
```

```
static double gM2 = 0.0;

// Установка шага округления значений функции и производной.
void SetRoundDelta(double Delta)
{
    gDelta = Delta;
}

// Установка констант m1 и M2 для критерия остановки.
void SetNewtonConstants(double m1, double M2)
{
    gM1 = m1;
    gM2 = M2;
}

// ==============================================================
// Функция Round - округление числа до заданной точности Delta
// ==============================================================
double Round(double X, double Delta)
{
    if (Delta <= 0.0) return X;
    if (Delta < 1E-12) {
        printf("Неверное задание точности округления\n");
        exit(1);
    }

    if (X >= 0.0)
        return Delta * floor(X / Delta + 0.5);
    else
        return Delta * ceil(X / Delta - 0.5);
}

// ==============================================================
// Метод Ньютона
// x_(n+1) = x_n - f(x_n) / f'(x_n)
// Критерий остановки согласно оценке погрешности:
```

```
// |x_n - x_(n-1)| < sqrt(2*m1*Eps/M2)
// ==============================================================
double NEWTON(double X, double Eps, int &N)
{
    // Проверка корректности входных данных.
    if (Eps <= 0.0) {
        printf("Неверное задание точности\n");
        exit(1);
    }
    if (gM1 <= 0.0 || gM2 <= 0.0) {
        printf("Не заданы константы m1 и M2 для метода Ньютона\n");
        exit(1);
    }

    // Порог остановки по априорной оценке погрешности метода Ньютона.
    double E0 = sqrt(2.0 * gM1 * fabs(Eps) / gM2);
    double XPrev = X;
    double FX, DFX, XNext;

    N = 0;

    // Основной итерационный процесс метода Ньютона.
    while (true) {
        FX = Round(F(XPrev), gDelta);
        DFX = Round(DF(XPrev), gDelta);

        // Производная в текущей точке не должна обращаться в нуль.
        if (DFX == 0.0) {
            printf("Производная равна нулю. Метод Ньютона
неприменим.\n");
            exit(1);
        }

        XNext = XPrev - FX / DFX;
        N++;
```

```
        // Критерий остановки по разности соседних приближений.
        if (fabs(XNext - XPrev) < E0)
            return XNext;

        // Новое приближение должно оставаться в области определения.
        if (XNext <= 0.0) {
            printf("Следующее приближение вышло из области
определения\n");
            exit(1);
        }

        // Защита от зацикливания при нарушении сходимости.
        if (N > 1000) {
            printf("Превышено допустимое число итераций\n");
            exit(1);
        }

        XPrev = XNext;
    }
}
```

Название файла: plot.py

```
import csv
import math
import matplotlib.pyplot as plt
import matplotlib.patheffects as pe

LEFT = 3.0
RIGHT = 4.0
X0 = 3.0


# ==============================================================
# Функция варианта 18
# ==============================================================
def f(x: float) -> float:
```

```
    if x <= 0.0:
        raise ValueError("x must be positive")
    return math.exp(1.0 / (x * x)) - math.log(x)


# ==============================================================
# Первая производная функции варианта 18
# ==============================================================
def df(x: float) -> float:
    if x <= 0.0:
        raise ValueError("x must be positive")
    return -2.0 * math.exp(1.0 / (x * x)) / (x ** 3) - 1.0 / x


# ==============================================================
# Один шаг метода Ньютона
# ==============================================================
def newton_step(x: float) -> float:
    return x - f(x) / df(x)


# ==============================================================
# График 0: функция и геометрический смысл метода Ньютона
# ==============================================================
xs = [2.2 + i * 0.005 for i in range(451)]
ys = [f(x) for x in xs]
x1 = newton_step(X0)
x2 = newton_step(x1)
root = newton_step(x2)

plt.figure(figsize=(8.2, 5.4))
plt.plot(xs, ys, color='navy', linewidth=2.2, label='f(x)', zorder=2)
plt.axhline(0.0, color='black', linewidth=1.0, zorder=1)
plt.axvspan(LEFT, RIGHT, alpha=0.09, label='отрезок [3;4]')

# Точки графика
```

```
plt.plot([LEFT, RIGHT], [f(LEFT), f(RIGHT)], 'o', markersize=8)
plt.plot([X0], [f(X0)], 'o', markersize=8)
plt.plot([x1, x2, root], [0.0, 0.0, 0.0], 'o', markersize=6)

# Касательная в x0
k0 = df(X0)
y0 = f(X0)
tx0 = [2.88, x1 + 0.025]
ty0 = [y0 + k0 * (tx0[0] - X0), y0 + k0 * (tx0[1] - X0)]
plt.plot(
    tx0,
    ty0,
    '--',
    color='crimson',
    linewidth=2.2,
    dashes=(8, 4),
    alpha=0.9,
    zorder=5,
    path_effects=[pe.Stroke(linewidth=3.6, foreground='white'),
pe.Normal()],
    label='касательная в x₀',
)

# Касательная в x1
k1 = df(x1)
y1 = f(x1)
tx1 = [x1 - 0.06, x2 + 0.05]
ty1 = [y1 + k1 * (tx1[0] - x1), y1 + k1 * (tx1[1] - x1)]
plt.plot(
    tx1,
    ty1,
    ':',
    color='darkorange',
    linewidth=2.4,
    alpha=0.9,
    zorder=5,
```

```
    path_effects=[pe.Stroke(linewidth=3.8, foreground='white'),
pe.Normal()],
    label='касательная в x₁',
)

plt.scatter(
    [X0, x1],
    [y0, y1],
    s=58,
    color=['crimson', 'darkorange'],
    edgecolors='white',
    linewidths=0.9,
    zorder=6,
)

# Вертикали
plt.vlines(X0, 0.0, y0, colors='gray', linestyles='dotted',
linewidth=0.9, alpha=0.9, zorder=1)
plt.vlines(x1, 0.0, y1, colors='gray', linestyles='dotted',
linewidth=0.9, alpha=0.9, zorder=1)

# Подписи
label_box = dict(boxstyle='round,pad=0.15', facecolor='white',
edgecolor='none', alpha=0.9)
plt.annotate('f(3)', (LEFT, f(LEFT)), xytext=(-22, 12),
textcoords='offset points', bbox=label_box, zorder=7)
plt.annotate('f(4)', (RIGHT, f(RIGHT)), xytext=(-12, -18),
textcoords='offset points', bbox=label_box, zorder=7)
plt.annotate('x₀ = 3', (X0, 0.0), xytext=(-4, -30), textcoords='offset
points', ha='center', bbox=label_box, zorder=7)
plt.annotate('x₁', (x1, 0.0), xytext=(-10, -14), textcoords='offset
points', ha='center', bbox=label_box, zorder=7)
plt.annotate('x₂', (x2, 0.0), xytext=(10, -31), textcoords='offset
points', ha='center', bbox=label_box, zorder=7)
plt.annotate('x*', (root, 0.0), xytext=(26, -8), textcoords='offset
points', ha='center', bbox=label_box, zorder=7)
```

```
plt.xlim(2.65, 4.15)
plt.ylim(-0.40, 0.12)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Функция f(x) и первые шаги метода Ньютона')
plt.grid(True, linestyle='--', alpha=0.55)
plt.legend(loc='lower left')
plt.savefig('newton_function_v18.png', dpi=220, bbox_inches='tight')
plt.close()

# ============================================================
# График 1: число итераций от точности Eps
# ============================================================
rows = []
with open('newton_iterations_vs_eps.csv', encoding='utf-8') as fcsv:
    reader = csv.DictReader(fcsv)
    for row in reader:
        rows.append((float(row['Eps']), int(row['Iterations'])))
rows.sort(key=lambda t: t[0])
eps = [r[0] for r in rows]
iters = [r[1] for r in rows]

plt.figure(figsize=(8.0, 5.0))
plt.semilogx(eps, iters, marker='o', linewidth=2.0)
plt.xlabel('Точность Eps')
plt.ylabel('Число итераций N')
plt.title('Сходимость метода Ньютона')
plt.grid(True, which='both', linestyle='--', alpha=0.55)
plt.savefig('newton_iterations_v18.png', dpi=220, bbox_inches='tight')
plt.close()

# ============================================================
# График 2: погрешность корня от точности округления Delta
# ============================================================
rows = []
```

```
with open('newton_sensitivity_vs_delta.csv', encoding='utf-8') as fcsv:
    reader = csv.DictReader(fcsv)
    for row in reader:
        d = float(row['Delta'])
        if d > 0.0:
            rows.append((d, float(row['Error'])))
rows.sort(key=lambda t: t[0])
delta = [r[0] for r in rows]
err = [r[1] for r in rows]

plt.figure(figsize=(8.0, 5.0))
plt.loglog(delta, err, marker='o', linewidth=2.0)
plt.xlabel('Точность округления Delta')
plt.ylabel('|x - x_ref|')
plt.title('Чувствительность метода Ньютона к округлению')
plt.grid(True, which='both', linestyle='--', alpha=0.55)
plt.savefig('newton_sensitivity_v18.png', dpi=220, bbox_inches='tight')
plt.close()
```